INFS3200 A1

Alex Donnellan (46963037)

Contents

1. Task 1

1.1. 1.A

```
SELECT COUNT(*) FROM employees;
```

```
emp_s4696303=# SELECT COUNT(*) FROM employees;
 count
--------
 300024
(1 row)
```

1.2. 1.B

```
SELECT COUNT(*) FROM dept_emp WHERE dept_no = (SELECT dept_no FROM departments WHERE
dept_name = 'Marketing');
```

```
emp_s4696303=# SELECT COUNT(*) FROM dept_emp WHERE dept_no = (SELECT dept_no FROM departments WHERE dept_name = 'Marketing');
 count
-------
 20211
(1 row)
```

2. Task 2

2.1. 2.A

```
CREATE TABLE IF NOT EXISTS salaries_h (
  emp_no int NOT NULL,
  salary int NOT NULL,
  from_date date NOT NULL,
  to_date date NOT NULL,
  PRIMARY KEY (emp_no, from_date),
  CONSTRAINT salaries_emp_no_fk FOREIGN KEY (emp_no) REFERENCES employees (emp_no)
) PARTITION BY RANGE (from_date);

CREATE TABLE salaries_1 PARTITION OF salaries_h FOR VALUES FROM (MINVALUE) TO
('1990-01-01');
CREATE TABLE salaries_2 PARTITION OF salaries_h FOR VALUES FROM ('1990-01-01') TO
('1992-01-01');
CREATE TABLE salaries_3 PARTITION OF salaries_h FOR VALUES FROM ('1992-01-01') TO
('1994-01-01');
```

```
CREATE TABLE salaries_4 PARTITION OF salaries_h FOR VALUES FROM ('1994-01-01') TO
('1996-01-01');
CREATE TABLE salaries_5 PARTITION OF salaries_h FOR VALUES FROM ('1996-01-01') TO
('1998-01-01');
CREATE TABLE salaries_6 PARTITION OF salaries_h FOR VALUES FROM ('1998-01-01') TO
('2000-01-01');
CREATE TABLE salaries_7 PARTITION OF salaries_h FOR VALUES FROM ('2000-01-01') TO
(MAXVALUE);

TRUNCATE TABLE salaries_h;
INSERT INTO salaries_h
SELECT * FROM salaries;
```

```
emp_s4696303=# CREATE TABLE IF NOT EXISTS salaries_h (
emp_s4696303(#   emp_no int NOT NULL,
emp_s4696303(#   salary int NOT NULL,
emp_s4696303(#   from_date date NOT NULL,
 ('1990-01-01') TO ('1992-01-01');
CREATE TABLE salaries_3 PARTITION OF salaries_h FOR VALUES FROM ('1992-01-01') TO ('1994-01-01');
CREATE TABLE salaries_4 PARTITION OF salaries_h FOR VALUES FROM ('1994-01-01') TO ('1996-01-01');
CREATE TABLE salaries_5 PARTITION OF salaries_h FOR VALUES FROM ('1996-01-01') TO ('1998-01-01');
CREATE TABLE salaries_6 PARTITION OF salaries_h FOR VALUES FROM ('1998-01-01') TO ('2000-01-01');
CREATE TABLE salariesemp_s4696303(#   to_date date NOT NULL,
emp_s4696303(#   PRIMARY KEY (emp_no, from_date),
emp_s4696303(#   CONSTRAINT salaries_emp_no_fk FOREIGN KEY (emp_no) REFERENCES employees (emp_no)
emp_s4696303(# ) PARTITION BY RANGE (from_date);
CREATE TABLE
emp_s4696303=#
emp_s4696303=# CREATE TABLE salaries_1 PARTITION OF salaries_h FOR VALUES FROM (MINVALUE) TO ('1990-01-01');
CREATE TABLE
emp_s4696303=# CREATE TABLE salaries_2 PARTITION OF salaries_h FOR VALUES FROM ('1990-01-01') TO ('1992-01-01');
CREATE TABLE
emp_s4696303=# CREATE TABLE salaries_3 PARTITION OF salaries_h FOR VALUES FROM ('1992-01-01') TO ('1994-01-01');
CREATE TABLE
emp_s4696303=# CREATE TABLE salaries_4 PARTITION OF salaries_h FOR VALUES FROM ('1994-01-01') TO ('1996-01-01');
CREATE TABLE
emp_s4696303=# CREATE TABLE salaries_5 PARTITION OF salaries_h FOR VALUES FROM ('1996-01-01') TO ('1998-01-01');
CREATE TABLE
emp_s4696303=# CREATE TABLE salaries_6 PARTITION OF salaries_h FOR VALUES FROM ('1998-01-01') TO ('2000-01-01');
CREATE TABLE
emp_s4696303=# CREATE TABLE salaries_7 PARTITION OF salaries_h FOR VALUES FROM ('2000-01-01') TO (MAXVALUE);
CREATE TABLE
emp_s4696303=#
emp_s4696303=# TRUNCATE TABLE salaries_h;
TRUNCATE TABLE
emp_s4696303=# INSERT INTO salaries_h
emp_s4696303-# SELECT * FROM salaries;
INSERT 0 2844047
emp_s4696303=#
```

2.2. 2.B

```
SELECT AVG(salary) FROM salaries_h WHERE from_date >= '1996-06-30' AND from_date <=
'1996-12-31';
```

```
emp_s4696303=# SELECT AVG(salary) FROM salaries_h WHERE from_date ⩾ '1996-06-30' AND from_date ⩽ '1996-12-31';
        avg
--------------------
 63724.924418447694
(1 row)
```

```
EXPLAIN SELECT AVG(salary) FROM salaries_h WHERE from_date >= '1996-06-30' AND
from_date <= '1996-12-31';
```

```
emp_s4696303=# EXPLAIN SELECT AVG(salary) FROM salaries_h WHERE from_date ⩾ '1996-06-30' AND from_date ⩽ '1996-12-31';
                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Finalize Aggregate  (cost=7589.38..7589.39 rows=1 width=32)
   →  Gather  (cost=7589.26..7589.37 rows=1 width=32)
         Workers Planned: 1
         →  Partial Aggregate  (cost=6589.26..6589.27 rows=1 width=32)
               →  Parallel Seq Scan on salaries_5 salaries_h  (cost=0.00..6424.81 rows=65780 width=4)
                     Filter: ((from_date ⩾ '1996-06-30'::date) AND (from_date ⩽ '1996-12-31'::date))
(6 rows)
```

From the plan we can see that postgres decides to make use of fragmentation by only scanning the single table that can contain the values, i.e. salaries_5. A full scan over this table occurs as there is no index on the from_date. The key choices postgres makes here to optimise and execute this plan are restricting to only the fragment that will contain data and donig a Parallel scan to allow checking multiple rows at a time.

2.3. 2.c

```
CREATE TABLE employees_public AS
SELECT emp_no, first_name, last_name, hire_date FROM employees;

ALTER TABLE employees_public ADD PRIMARY KEY (emp_no);

CREATE TABLE employees_confidential as
SELECT emp_no, birth_date, gender FROM employees;

/* Export employees_confidential table */

CREATE DATABASE emp_confidential;

/* Connect to emp_confidential & load employees_confidential */

DROP TABLE employees_confidential;
```

```
emp_s4696303=# CREATE TABLE employees_public AS
emp_s4696303-# SELECT emp_no, first_name, last_name, hire_date FROM employees;
SELECT 300024
emp_s4696303=#
emp_s4696303=# ALTER TABLE employees_public ADD PRIMARY KEY (emp_no);
ALTER TABLE
emp_s4696303=# CREATE TABLE employees_confidential as
emp_s4696303-# SELECT emp_no, birth_date, gender FROM employees;
SELECT 300024
emp_s4696303=# CREATE DATABASE emp_confidential;
CREATE DATABASE
emp_s4696303=# ^C
emp_s4696303=# \q
s4696303@infs3200-0b490d8e:~/infs3200/a1$ pg_d
pg_dropcluster  pg_dump         pg_dumpall
s4696303@infs3200-0b490d8e:~/infs3200/a1$ pg_dump -d emp_s4696303 -t employees_confidential > employees_confidential.sql
s4696303@infs3200-0b490d8e:~/infs3200/a1$ psql
psql (17.7 (Ubuntu 17.7-3.pgdg24.04+1))
Type "help" for help.

s4696303=# \c employ

s4696303=# \c emp_confidential
You are now connected to database "emp_confidential" as user "s4696303".
emp_confidential=# \i employees_confidential.sql
SET
SET
SET
SET
SET
SET
 set_config
------------

(1 row)

SET
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
COPY 300024
emp_confidential=# \c emp_s4696303
You are now connected to database "emp_s4696303" as user "s4696303".
emp_s4696303=# DROP TABLE employees_confidential;
DROP TABLE
emp_s4696303=#
```

3. Task 3

3.1. 3.a

Lets consider each replication strategy. First no replication, this means that each emp_no tuple is only stored at a single (usually what would be considered the ‘best’ site for each emp_no) site. This has a few key benefits. Namely Decreased storage costs, easier updates as you only need to update one site and much easier to maintain a single source of truth. However it also comes with some risks, namely

the site becomes a single point of failure for the emp_no, if it goes down the data becomes unavailable, costly remote data operations for joints and unions.

Secondly full replication, this means that each site contains all tuples. This provides superior performance over the no replication strategy as there is no extra network hop needed to collect data from other sites, the DDBMS also becomes much more reliable/available as data is replicated to all sites, meaning one going down doesn't make some data unavailable. It can also allow the DDBMS to have functional 'read replicas' allow for load to be shared between servers. This comes at the cost of increased storage requirements, update issues as updates need to propagate to all sites leading to increased write latency.

Finally partial replication is a combination of the prior two approaches. It overs similar pros to both, read load can be shared between servers, lower data storage costs than full repliation and increased availability/reliabilty than no repication. However it does still face similar update propagation issues as full replication, with the addition of needing to manage which sites data is replicated.

3.2. 3.b

Each strategy has a different process, no replication being the simplist as the query planner simply needs to determine which site the tuple is stored at and update it at that site. Full replication is next with ease of update, the planner needs to perfrom the update on all sites, this would however be the slowest update process to ensure consistency across sites. Finally partial replication requires the planner to determine which sites contain the record and tell just them to update, this would sit between no and full replication in terms of speed of update.

4. Task 4

4.1. 4.a

```
Create extension postgres_fdw;

Create server foreign_server Foreign data wrapper postgres_fdw OPTIONS (host 'infs3200-
sharedb.zones.eait.uq.edu.au', port '5432', dbname 'sharedb');

create user mapping for "s4696303" server foreign_server options (user 'sharedb',
password 'Y3Y7FdqDSM9.3d47XUWg');

Create foreign table titles (
  emp_no integer NOT NULL,
    title VARCHAR NOT NULL,
    from_date date NOT NULL,
    to_date date NOT NULL)
  server foreign_server options (schema_name 'public', table_name 'titles');
```

```
emp_s4696303=# Create extension postgres_fdw;
CREATE EXTENSION
<dw OPTIONS (host 'infs3200-sharedb.zones.eait.uq.edu.au', port '5432', dbname 'sharedb');
CREATE SERVER
emp_s4696303=# create user mapping for "s4696303" server foreign_server options (user 'sharedb', password 'Y3Y7FdqDSM9.3d47XUWg');
CREATE USER MAPPING
emp_s4696303=# Create foreign table titles (
emp_s4696303(# emp_no integer NOT NULL,
emp_s4696303(#     title VARCHAR NOT NULL,
emp_s4696303(#     from_date date NOT NULL,
emp_s4696303(#     to_date date NOT NULL)
emp_s4696303-# server foreign_server options (schema_name 'public', table_name 'titles');
CREATE FOREIGN TABLE
```

4.2. 4.b

```sql
SELECT AVG(salary), t.title FROM salaries s
INNER JOIN employees e ON s.emp_no = e.emp_no
INNER JOIN titles t ON t.emp_no = s.emp_no
WHERE t.to_date = '9999-01-01'
GROUP BY t.title;
```

```
emp_s4696303=# SELECT AVG(salary), t.title FROM salaries s
emp_s4696303-# INNER JOIN employees e ON s.emp_no = e.emp_no
emp_s4696303-# INNER JOIN titles t ON t.emp_no = s.emp_no
emp_s4696303-# WHERE t.to_date = '9999-01-01'
emp_s4696303-# GROUP BY t.title;

        avg          |       title
---------------------+--------------------
 55372.205637022085 | Engineer
 60811.905380840108 | Senior Engineer
 64969.244604316547 | Manager
 53386.235589743590 | Assistant Engineer
 63346.256308081825 | Staff
 70720.308873766553 | Senior Staff
 59744.074925382562 | Technique Leader
(7 rows)
```

4.3. 4.c

First we setup the FDW table

```sql
-- Create extension postgres_fdw;

CREATE USER emp_readonly_user WITH PASSWORD 'infs3200';

\c emp_confidential

GRANT CONNECT ON DATABASE emp_confidential TO emp_readonly_user;

GRANT USAGE ON SCHEMA public TO emp_readonly_user;
```

```
GRANT SELECT ON TABLE employees_confidential TO emp_readonly_user;

\c emp_s4696303

Create server foreign_server_emp Foreign data wrapper postgres_fdw OPTIONS (host
'localhost', port '5432', dbname 'emp_confidential');

create user mapping for "s4696303" server foreign_server_emp options (user
'emp_readonly_user', password 'infs3200');

Create foreign table employees_confidential (
  emp_no integer NOT NULL,
    birth_date date NOT NULL,
    gender varchar NOT NULL)
  server foreign_server_emp options (schema_name 'public', table_name
'employees_confidential');
```

```
emp_s4696303=# GRANT SELECT ON TABLE customers TO readonly_user;
ERROR:  relation "customers" does not exist
emp_s4696303=# CREATE USER emp_readonly_user WITH PASSWORD 'infs3200';
CREATE ROLE
emp_s4696303=# \c emp_confidential
You are now connected to database "emp_confidential" as user "s4696303".
emp_confidential=# GRANT CONNECT ON DATABASE “emp_confidential” TO emp_readonly_user;
ERROR:  database ""“emp_confidential”"" does not exist
emp_confidential=# GRANT CONNECT ON DATABASE emp_confidential TO emp_readonly_user;
GRANT
emp_confidential=# GRANT USAGE ON SCHEMA public TO emp_readonly_user;
GRANT
emp_confidential=# GRANT SELECT ON TABLE emp_confidential TO emp_readonly_user;
ERROR:  relation "emp_confidential" does not exist
emp_confidential=# GRANT SELECT ON TABLE emp_confidential TO emp_readonly_user;
ERROR:  relation "emp_confidential" does not exist
emp_confidential=# GRANT SELECT ON TABLE employees_confidential TO emp_readonly_user;
GRANT
< wrapper postgres_fdw OPTIONS (host 'localhost', port '5432', dbname 'emp_confidential');
ERROR:  foreign-data wrapper "postgres_fdw" does not exist
emp_confidential=# \\c emp_s4696303
invalid command \
Try \? for help.
emp_confidential=# \c emp_s4696303
You are now connected to database "emp_s4696303" as user "s4696303".
<ta wrapper postgres_fdw OPTIONS (host 'localhost', port '5432', dbname 'emp_confidential');
ERROR:  server "foreign_server" already exists
< wrapper postgres_fdw OPTIONS (host 'localhost', port '5432', dbname 'emp_confidential');
ERROR:  server "foreign_server" already exists
< wrapper postgres_fdw OPTIONS (host 'localhost', port '5432', dbname 'emp_confidential');
CREATE SERVER
emp_s4696303=# create user mapping for "s4696303" server foreign_server_emp options (user 'emp_readonly_user', password 'infs3200');
CREATE USER MAPPING
emp_s4696303=# Create foreign table titles (
emp_s4696303(# emp_no integer NOT NULL,
emp_s4696303(#     birthdate date NOT NULL,
emp_s4696303(#     gender enum NOT NULL)
emp_s4696303-# server foreign_server_emp options (schema_name 'public', table_name 'employees_confidential');
ERROR:  type "enum" does not exist
LINE 4:     gender enum NOT NULL)
                   ^
emp_s4696303=#
emp_s4696303=# Create foreign table titles (
emp_s4696303(# emp_no integer NOT NULL,
emp_s4696303(#     birthdate date NOT NULL,
emp_s4696303(#     gender varchar NOT NULL)
emp_s4696303-# server foreign_server_emp options (schema_name 'public', table_name 'employees_confidential');
ERROR:  relation "titles" already exists
emp_s4696303=# Create foreign table emp_confidential (
emp_s4696303(# emp_no integer NOT NULL,
emp_s4696303(#     birthdate date NOT NULL,
emp_s4696303(#     gender varchar NOT NULL)
emp_s4696303-# server foreign_server_emp options (schema_name 'public', table_name 'employees_confidential');
CREATE FOREIGN TABLE
```

```
emp_s4696303=# Create foreign table employees_confidential (
emp_s4696303(# emp_no integer NOT NULL,
emp_s4696303(#     birth_date date NOT NULL,
emp_s4696303(#     gender varchar NOT NULL)
emp_s4696303-# server foreign_server_emp options (schema_name 'public', table_name 'employees_confidential');
CREATE FOREIGN TABLE
```

Now we can perform the semi join

```
SELECT first_name, last_name FROM employees_public eps,
(SELECT ec.emp_no, ec.birth_date FROM employees_confidential ec, (SELECT emp_no FROM
employees_public) ep WHERE ec.emp_no = ep.emp_no) fr
WHERE fr.emp_no = eps.emp_no AND fr.birth_date >= '1970-01-01' AND fr.birth_date <=
'1975-01-01';
```

```
emp_s4696303=# SELECT first_name, last_name FROM employees_public eps,
emp_s4696303-# (SELECT ec.emp_no, ec.birth_date FROM employees_confidential ec, (SELECT emp_no FROM employees_public) ep WHERE ec.emp_no = ep.emp_no) fr
emp_s4696303-# WHERE fr.emp_no = eps.emp_no AND fr.birth_date ⩾ '1970-01-01' AND fr.birth_date ⩽ '1975-01-01';
 first_name | last_name
------------+-----------
(0 rows)
```

however there are no results returned, we can validate that this is try by checking the employees confidential table and validating against the existing employees table

```
emp_confidential=# SELECT * FROM employees_confidential WHERE birth_date ⩾ '1970-01-01' AND birth_date ⩽ '1975-01-01';
 emp_no | birth_date | gender
--------+------------+--------
(0 rows)

emp_confidential=# \c emp_s4696303
You are now connected to database "emp_s4696303" as user "s4696303".
emp_s4696303=# SELECT * FROM employees_confidential WHERE birth_date ⩾ '1970-01-01' AND birth_date ⩽ '1975-01-01';
 emp_no | birth_date | gender
--------+------------+--------
(0 rows)

emp_s4696303=# SELECT * FROM employees WHERE birth_date ⩾ '1970-01-01' AND birth_date ⩽ '1975-01-01';
 emp_no | birth_date | first_name | last_name | gender | hire_date
--------+------------+------------+-----------+--------+-----------
(0 rows)
```

4.4. 4.d

For an inner-join we directly send all necessary attributes from the remote site to the local site

```
SELECT first_name, last_name FROM employees eps
INNER JOIN employees_confidential ec ON ec.emp_no = eps.emp_no
WHERE ec.birth_date >= '1970-01-01' AND ec.birth_date <= '1975-01-01';
```

```
emp_s4696303=# SELECT first_name, last_name FROM employees eps
emp_s4696303-# INNER JOIN employees_confidential ec ON ec.emp_no = eps.emp_no
emp_s4696303-# WHERE ec.birth_date ⩾ '1970-01-01' AND ec.birth_date ⩽ '1975-01-01';
 first_name | last_name
------------+-----------
(0 rows)
```

In this case the semi-join will have a higher transmission cost as it has to transmit the initial emp_no to match and transmit back the emp_no & their respecitve birthdates. It could be switched around by mooving the birth_date filter condtion inside the semi-join so that it occurs at the remote site, like so

```
SELECT first_name, last_name FROM employees_public eps,
(SELECT ec.emp_no, ec.birth_date FROM employees_confidential ec, (SELECT emp_no FROM
employees_public) ep WHERE ec.emp_no = ep.emp_no
AND ec.birth_date >= '1970-01-01' AND ec.birth_date <= '1975-01-01') fr
WHERE fr.emp_no = eps.emp_no;
```

The above (for our current data set) significantly reduces the transmission cost as no rows match the predicate so only minimal information is needed for the transmission.